CODE :

```
CREATE OR REPLACE WAREHOUSE hacker
WAREHOUSE_SIZE = 'XSMALL'
AUTO_SUSPEND = 60
AUTO_RESUME = TRUE
INITIALLY_SUSPENDED = TRUE;

CREATE OR REPLACE DATABASE GROCARY_DB;

USE DATABASE GROCARY_DB;

CREATE OR REPLACE SCHEMA RAW;
CREATE OR REPLACE SCHEMA SILVER;
CREATE OR REPLACE SCHEMA DWH;
use schema raw;
CREATE OR REPLACE STAGE grocary_stage;



//raw tables//
CREATE OR REPLACE TABLE RAW.raw_stores (
   store_id INT,
   store_name STRING,
   city STRING,
   state STRING,
   region STRING,
   store_type STRING,
```

```
  opening_date DATE,
  size_sqft INT,
  status STRING
);

CREATE OR REPLACE TABLE RAW.raw_products (
  product_id INT,
  category STRING,
  subcategory STRING,
  brand STRING,
  product_name STRING,
  unit_size STRING,
  mrp FLOAT,
  base_price FLOAT,
  status STRING
);

CREATE OR REPLACE TABLE RAW.raw_inventory (
  inventory_id INT,
  store_id INT,
  product_id INT,
  on_hand_qty INT,
  safety_stock_qty INT,
  last_restock_date DATE,
  last_count_date DATE
);

CREATE OR REPLACE TABLE RAW.raw_sales (
```

```
  sale_id INT,
  store_id INT,
  product_id INT,
  timestamp TIMESTAMP,
  quantity_sold INT,
  unit_price FLOAT,
  discount FLOAT,
  total_amount FLOAT,
  payment_mode STRING
);

CREATE OR REPLACE TABLE RAW.raw_promotions (
  promo_id INT,
  product_id INT,
  store_id INT,
  promo_type STRING,
  start_date DATE,
  end_date DATE,
  discount_pct FLOAT,
  is_active BOOLEAN
);



CREATE OR REPLACE FILE FORMAT my_csv_format
TYPE = 'CSV'
FIELD_OPTIONALLY_ENCLOSED_BY = '"'
SKIP_HEADER = 1;
```

```
-- MASTER

COPY INTO RAW.raw_stores FROM @grocary_stage/stores_master.csv
FILE_FORMAT=my_csv_format;

COPY INTO RAW.raw_products FROM @grocary_stage/products_master.csv
FILE_FORMAT=my_csv_format;

COPY INTO RAW.raw_inventory FROM @grocary_stage/inventory_master.csv
FILE_FORMAT=my_csv_format;

COPY INTO RAW.raw_sales FROM @grocary_stage/sales_master.csv
FILE_FORMAT=my_csv_format;

COPY INTO RAW.raw_promotions FROM @grocary_stage/promotions_master.csv
FILE_FORMAT=my_csv_format;


-- INCREMENTAL

COPY INTO RAW.raw_stores FROM @grocary_stage/stores_inc.csv
FILE_FORMAT=my_csv_format;

COPY INTO RAW.raw_products FROM @grocary_stage/products_inc.csv
FILE_FORMAT=my_csv_format;

COPY INTO RAW.raw_inventory FROM @grocary_stage/inventory_inc.csv
FILE_FORMAT=my_csv_format;

COPY INTO RAW.raw_sales FROM @grocary_stage/sales_inc.csv
FILE_FORMAT=my_csv_format;

COPY INTO RAW.raw_promotions FROM @grocary_stage/promotions_inc.csv
FILE_FORMAT=my_csv_format;


//validation layer//

CREATE OR REPLACE TABLE RAW.validated_stores AS

SELECT DISTINCT * FROM RAW.raw_stores WHERE store_id IS NOT NULL;
```

```
CREATE OR REPLACE TABLE RAW.validated_products AS
SELECT DISTINCT * FROM RAW.raw_products WHERE product_id IS NOT NULL;

CREATE OR REPLACE TABLE RAW.validated_inventory AS
SELECT * FROM RAW.raw_inventory WHERE on_hand_qty >= 0;

CREATE OR REPLACE TABLE RAW.validated_sales AS
SELECT * FROM RAW.raw_sales
WHERE quantity_sold >= 0 AND unit_price >= 0;

CREATE OR REPLACE TABLE RAW.validated_promotions AS
SELECT * FROM RAW.raw_promotions
WHERE discount_pct BETWEEN 0 AND 100 AND start_date <= end_date;


//silver layer//
USE SCHEMA SILVER;

CREATE OR REPLACE TABLE dim_products AS
SELECT DISTINCT * FROM RAW.validated_products;

CREATE OR REPLACE TABLE dim_stores AS
SELECT DISTINCT * FROM RAW.validated_stores;

CREATE OR REPLACE TABLE inventory_clean AS
SELECT i.*
FROM RAW.validated_inventory i
```

```
JOIN dim_products p ON i.product_id = p.product_id
JOIN dim_stores s ON i.store_id = s.store_id;

CREATE OR REPLACE TABLE sales_clean AS
SELECT s.*
FROM RAW.validated_sales s
JOIN dim_products p ON s.product_id = p.product_id
JOIN dim_stores st ON s.store_id = st.store_id;

CREATE OR REPLACE TABLE promotions_clean AS
SELECT pr.*
FROM RAW.validated_promotions pr
JOIN dim_products p ON pr.product_id = p.product_id;


//DWH layer fact table with merge//
USE SCHEMA DWH;

CREATE OR REPLACE TABLE fact_sales LIKE SILVER.sales_clean;

MERGE INTO fact_sales t
USING SILVER.sales_clean s
ON t.sale_id = s.sale_id
WHEN MATCHED THEN UPDATE SET
  quantity_sold = s.quantity_sold,
  unit_price = s.unit_price,
  total_amount = s.total_amount
WHEN NOT MATCHED THEN
```

```
INSERT VALUES (
   s.sale_id, s.store_id, s.product_id,
   s.timestamp, s.quantity_sold,
   s.unit_price, s.discount,
   s.total_amount, s.payment_mode
);

CREATE OR REPLACE TABLE fact_inventory AS
SELECT * FROM SILVER.inventory_clean;

CREATE OR REPLACE TABLE fact_promotions AS
SELECT * FROM SILVER.promotions_clean;



//scd type2//
CREATE OR REPLACE TABLE dim_products_scd (
   product_id INT,
   category STRING,
   subcategory STRING,
   brand STRING,
   product_name STRING,
   start_date DATE,
   end_date DATE,
   is_current BOOLEAN
);
INSERT INTO dim_products_scd
SELECT
   product_id, category, subcategory, brand, product_name,
```

```
  CURRENT_DATE, NULL, TRUE
FROM SILVER.dim_products;



select * from dim_products_scd;
MERGE INTO dim_products_scd tgt
USING SILVER.dim_products src
ON tgt.product_id = src.product_id AND tgt.is_current = TRUE

WHEN MATCHED AND (
  tgt.category != src.category OR
  tgt.brand != src.brand OR
  tgt.subcategory != src.subcategory
)
THEN UPDATE SET
  end_date = CURRENT_DATE,
  is_current = FALSE

WHEN NOT MATCHED THEN
INSERT VALUES (
  src.product_id, src.category, src.subcategory,
  src.brand, src.product_name,
  CURRENT_DATE, NULL, TRUE
);

//anomoly detection//
CREATE OR REPLACE TABLE anomaly_stockout AS
SELECT s.*
```

```
FROM fact_sales s
JOIN fact_inventory i
ON s.product_id = i.product_id AND s.store_id = i.store_id
WHERE i.on_hand_qty = 0;

CREATE OR REPLACE TABLE anomaly_price_drop AS
SELECT s.*
FROM fact_sales s
JOIN SILVER.dim_products p
ON s.product_id = p.product_id
WHERE s.unit_price < 0.7 * p.base_price;

CREATE OR REPLACE TABLE anomaly_promo_overlap AS
SELECT p1.*
FROM fact_promotions p1
JOIN fact_promotions p2
ON p1.product_id = p2.product_id
AND p1.store_id = p2.store_id
AND p1.promo_id != p2.promo_id
WHERE p1.start_date <= p2.end_date
AND p2.start_date <= p1.end_date;

//KPI metrices//
-- Stockout Rate
SELECT
(COUNT(CASE WHEN on_hand_qty = 0 THEN 1 END)*100.0)/COUNT(*) AS
stockout_rate
FROM fact_inventory;
```

```
-- Inventory Turnover
SELECT
SUM(quantity_sold) / NULLIF(AVG(on_hand_qty),0) AS inventory_turnover
FROM fact_sales s
JOIN fact_inventory i ON s.product_id = i.product_id;

-- Category Sales
SELECT p.category,
SUM(s.quantity_sold * s.unit_price) AS total_sales
FROM fact_sales s
JOIN SILVER.dim_products p
ON s.product_id = p.product_id
GROUP BY p.category;

-- Avg Discount
SELECT AVG(discount) AS avg_discount,
SUM(total_amount) AS total_sales
FROM fact_sales;

-- Revenue per Store
SELECT store_id,
SUM(total_amount) AS revenue
FROM fact_sales
GROUP BY store_id;

//stream and task//
CREATE OR REPLACE STREAM sales_stream ON TABLE RAW.raw_sales;
```

```
CREATE OR REPLACE TASK sales_task
WAREHOUSE = hack
SCHEDULE = '5 MINUTE'
AS
MERGE INTO fact_sales t
USING sales_stream s
ON t.sale_id = s.sale_id
WHEN MATCHED THEN UPDATE SET quantity_sold = s.quantity_sold
WHEN NOT MATCHED THEN
INSERT VALUES (
  s.sale_id, s.store_id, s.product_id,
  s.timestamp, s.quantity_sold,
  s.unit_price, s.discount,
  s.total_amount, s.payment_mode
);

ALTER TASK sales_task RESUME;

CREATE OR REPLACE TABLE dwh.kpi_stockout AS
SELECT
(COUNT(CASE WHEN on_hand_qty = 0 THEN 1 END)*100.0) / NULLIF(COUNT(*), 0) AS stockout_rate
FROM dwh.fact_inventory;

CREATE OR REPLACE TABLE dwh.kpi_stockout AS
SELECT
(COUNT(CASE WHEN on_hand_qty = 0 THEN 1 END)*100.0) / NULLIF(COUNT(*), 0) AS stockout_rate
```

```
FROM dwh.fact_inventory;


CREATE OR REPLACE TABLE dwh.kpi_category_sales AS
SELECT
p.category,
SUM(s.total_amount) AS total_sales
FROM dwh.fact_sales s
JOIN silver.dim_products p
ON s.product_id = p.product_id
GROUP BY p.category;


CREATE OR REPLACE TABLE dwh.kpi_store_revenue AS
SELECT
store_id,
SUM(total_amount) AS revenue
FROM dwh.fact_sales
GROUP BY store_id;


CREATE OR REPLACE TABLE dwh.kpi_discount AS
SELECT
AVG(discount) AS avg_discount,
SUM(total_amount) AS total_sales
FROM dwh.fact_sales;
```

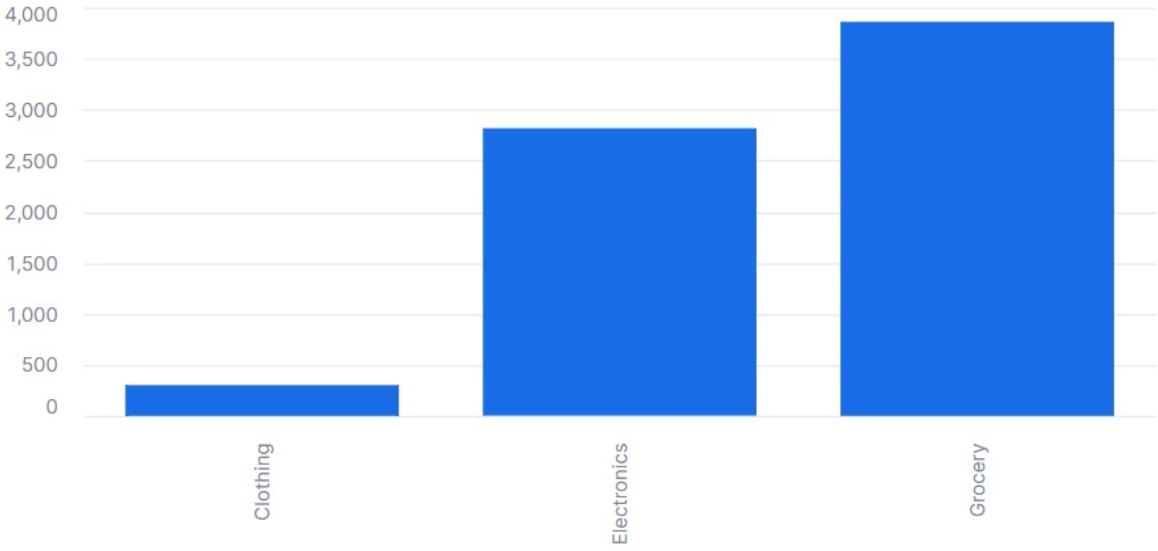
Outputs :

🛒 Retail Analytics Dashboard

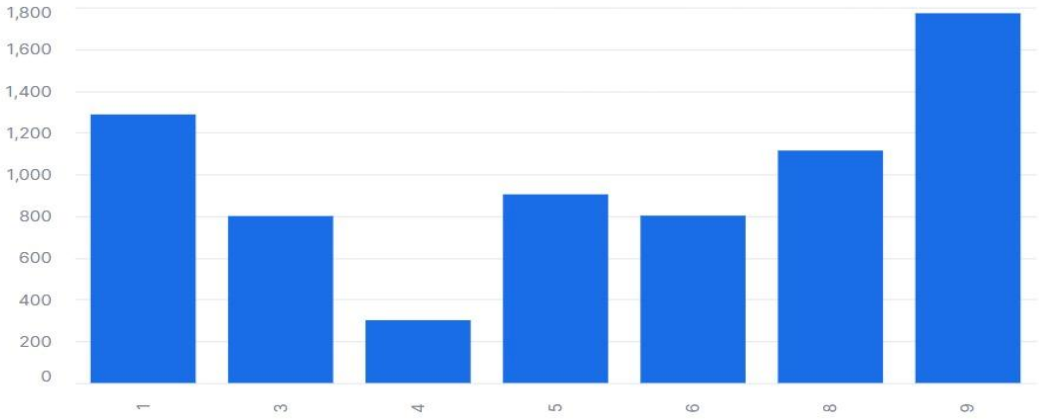
Stockout Rate (%)

0.0

📊 Category Sales



🏪 Revenue per Store



Average Discount

19.23

Total Revenue

6980.0